

PROG1: Naming Conventions & C++ Core Guidelines

Selected rules for IDAE GD PROG1 and additional naming conventions.

All core guidelines: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines!>

*The selected Core C++ Guidelines in this document are **not** just course-specific rules.*

They are part of the official C++ Core Guidelines, edited and maintained by none other than

Bjarne Stroustrup (creator of C++) and Herb Sutter, and are widely regarded as the industry's best-practice guidelines for modern C++.

PROG1: Naming conventions

Addition to core guideline [NL.8](#): naming conventions enforced in Programming 1 to improve readability, **consistency**, and communication within the course.

Category	Convention / Pattern	Code Example
Local Variables	lowerCamelCasing	<code>float width{};</code> <code>const float innerSize{ 10.f };</code>
Global Variables	g_ UpperCamelCasing	<code>int g_NumberOfEnemies{};</code> <code>const float g_WindowWidth{ 500.0f };</code>
Boolean Identifiers	Verb indicating boolean state (Is... , Has...)	<code>bool isReady{};</code> <code>bool g_IsAlive{};</code> <code>bool IsKeyPressed();</code>
Functions / Methods	UpperCamelCasing (Verb-based)	<code>ClearBackground();</code> // return type void <code>GetRandomNumber();</code> // returns a value
Function Parameters	lowerCamelCasing	<code>void DrawDonut(float centerX, float centerY,</code> <code>float innerRadius, float outerRadius);</code>
Types (Struct, Enum)	UpperCamelCasing	<code>struct Circle{...};</code> <code>enum class LightState{...};</code>
Enum Values	lowerCamelCasing	<code>enum class Direction { none, leftUp, rightDown };</code>
Pointers <i>Note: This naming convention temporarily deviates from the C++ Core Guideline NL5 for educational purposes.</i>	Use prefix p_ , then UpperCamelCasing	<code>int* pNumbers{};</code> <code>Enemy* m_pEnemy{};</code>

F: Functions

A function should do one clear job. Name it, size it, and pass its arguments so the caller can't misuse it.

Rule & Explanation	Code Example	Core Guideline Ref & URL
F.1; Name meaningful operations If a chunk of code does one well-defined thing, pull it into a named function instead of leaving it inline.	<pre>// bad: must inspect all code to understand purpose: int main() { std::cout << "Level: 12\n"; std::cout << "Health: 85\n"; } // good: clear function name, reusable, readable: void PrintPlayerStats() { std::cout << "Level: 12\n"; std::cout << "Health: 85\n"; } int main() { PrintPlayerStats(); }</pre>	F.1: Package operations as functions
F.3; Keep functions short and simple One-to-five-line functions are normal. If it doesn't fit on a screen, split it into smaller named pieces. Break large functions up into smaller cohesive and named functions.	<pre>void StartGame() { LoadLevel(); SpawnPlayer(); }</pre>	F.3: Keep functions short and simple
F.16; Pass cheap types by value, others by const& Small types (int, double, pointers) pass by value. Larger types (string, vector) pass by reference to const.	<pre>void f1(const std::string& s); // good: by const& void f4(int x); // good: int by value void f2(std::string s); // bad: large copy void f3(Color4f color); // bad: large copy</pre>	F.16: Passing "in" parameters
F.20; Prefer return values to output parameters Return the result instead of writing it into a parameter if possible; it's clearer and enables move semantics.	<pre>// bad void CalculateResult(int& out); // good int CalculateResult();</pre>	F.20: Prefer return values

Enum: Enumerations

Enumerations name a fixed set of related values. Use them instead of loose integers or macros.

Rule & Explanation	Code Example	Core Guideline Ref & URL
Enum.1; Prefer enumerations over macros A #define constant has no type and no scope. An enum is type-checked and scoped.	<pre>#define RED 0 // bad #define GREEN 1 enum class Color { red, green }; // good</pre>	Enum.1: Prefer enumerations over macros
Enum.3; Prefer class enums over plain enums A scoped enum class avoids leaking its enumerator names into the surrounding scope and avoids silent conversions to int.	<pre>enum Color { red, green }; // bad: leaks names enum class Color { red, green }; // good Color fill { Color::red }; // good: clear, readable</pre>	Enum.3: Prefer class enums
Enum.5; Don't use ALL_CAPS for enumerators ALL_CAPS collides with the convention for macros. Use normal identifier casing for enumerators.	<pre>enum class Color { RED, GREEN }; // bad enum class Color { red, green }; // good: lowerCamelCasing</pre>	Enum.5: Don't ALL CAPS enumerators

ES: Expressions and Statements

Write statements and expressions that say clearly what they do, and that the compiler can check.

Rule & Explanation	Code Example	Core Guideline Ref & URL
ES. 10; Declare one name (only) per declaration One declaration per line increases readability and avoids mistakes related to the C/C++ grammar.	<pre>int a = 10, b, c = 12; // bad: hard to read char *p, c; // bad: hard to interpret</pre>	ES. 10: Declare one name (only) per declaration
ES.20; Always initialize an object An uninitialized variable is undefined behavior waiting to happen. Give every variable a starting value.	<pre>int x; // bad: uninitialized int score{}; // good: default int value float ratio{1.5f}; // good</pre>	ES.20: Always initialize an object
ES.23; Prefer the {} initializer syntax Brace initialization is uniform, works everywhere, and blocks silent narrowing conversions.	<pre>int count = 3.14; // bad: silent truncation int count{10}; // good</pre>	ES.23: Prefer {} initializer syntax
ES.47; Use nullptr, not 0 or NULL nullptr has a real pointer type, so it can't be confused with the integer 0 in overload resolution.	<pre>int* p1{NULL}; // bad int* p2{nullptr}; // good p3 = nullptr; // good</pre>	ES.47: Use nullptr
ES.48; Avoid casts A cast overrides the type system's checks. Prefer code where no cast is needed or use a named cast if one is unavoidable.	<pre>double d{3.9}; int i1{(int)d}; // bad: C-style cast int i2{static_cast<int>(d)}; // better, if needed</pre>	ES.48: Avoid casts

Per: Performance

Performance work is only worth doing when there's a real, measured problem to fix.

Rule & Explanation	Code Example	Core Guideline Ref & URL
Per.1; Don't optimize without a reason Optimizing code that isn't a bottleneck just adds complexity and risk for no benefit.	<pre>// don't rewrite this into unreadable // bit-tricks unless it's proven to be slow</pre>	Per.1: Don't optimize without reason
Per.2; Don't optimize prematurely Get a correct, clear version working first. Optimize afterward, only where measurement shows it's needed.	<pre>// write the clear version first vector<int> v = compute();</pre>	Per.2: Don't optimize prematurely
Per.6; Don't claim performance without measuring "This should be faster" is a guess. Profile before and after any performance-motivated change.	<pre>// use a profiler / benchmark, // don't guess which version is faster</pre>	Per.6: Measure, don't guess

Con: Constants and Immutability

A value that can't change is easier to reason about and safer to share. Make things const by default.

Rule & Explanation	Code Example	Core Guideline Ref & URL
Con.1; By default, make objects immutable Declare a variable const unless you have a concrete reason it must change later.	<pre>int x{4}; // bad if x never changes const int x2{4}; // good const int g_MaxScore{10}; // good</pre>	Con.1: Make objects immutable by default

SF: Source Files

Keep declarations and definitions organized across .h and .cpp files so builds stay fast and consistent.

Rule & Explanation	Code Example	Core Guideline Ref & URL
SF.1; .cpp for code, .h for interfaces Use a consistent suffix convention: implementation goes in .cpp, declarations shared across files go in .h.	<pre>// Widget.h -> class/function declarations // Widget.cpp -> function bodies</pre>	SF.1: .cpp / .h convention
SF.5; A .cpp file must include its own .h Including your own header lets the compiler catch mismatches between declaration and definition immediately.	<pre>// Widget.cpp #include "Widget.h" // good: catches mismatches</pre>	SF.5: .cpp includes its own .h

NL: Naming and Layout

Naming and layout rules don't change behavior, but a consistent style makes code much faster to read.

Rule & Explanation	Code Example	Core Guideline Ref & URL
NL.1; Don't say in comments what the code already says A comment that just restates the code adds noise and can go stale. Comment on why, not on what, when the what is already obvious.	<pre>x = x + 1; // increment x // bad: restates code x = x + 1; // compensate for off-by-one // good</pre>	NL.1: Avoid redundant comments
NL.2; State intent in comments A comment should explain what a piece of code is trying to achieve, so a reader can check that the code matches the intent.	<pre>// good: explains purpose // Offset required to align texture with border grid float offset{ borderPadding * 2.0f };</pre>	NL.2: State intent in comments
NL.3; Keep comments crisp Write comments with the same care as code: clear, consistent grammar, spelling, and punctuation.	<pre>// bad: rambling, unclear // this here basically does the thing for x i guess // good: crisp // clamps x to [0, 100]</pre>	NL.3: Keep comments crisp
NL.4; Maintain a consistent indentation style Pick one indentation style for a project and stick to it; mixed styles make code harder to scan.	<pre>if (x) { f1(); } else { // bad: inconsistent with { after if f2(); // bad: inconsistent indentation }</pre>	NL.4: Consistent indentation
NL.5; Don't encode type information in names Avoid Hungarian-notation-style prefixes (like iCount); the type system and IDE already track types for you.	<pre>int iCount{}; // bad: type in name int countInt{}; // bad: type in name int count{}; // good</pre>	NL.5: Don't encode types in names
NL.8; Use a consistent naming style Identifiers should follow one clear, consistent style across the codebase (e.g. always camelCase or always snake_case).	<pre>int scoreLimit{}; // good: clear, consistent void ProcessInput(); // good: clear intent int a{}; // bad: unclear meaning void DoStuff(); // bad: unclear intent void do_stuff(); // bad: inconsistent in PROG1</pre>	NL.8: Use a consistent naming style
NL.9; Use ALL_CAPS only for macros Reserve ALL_CAPS names for macros, so readers instantly know an ALL_CAPS identifier is a preprocessor macro, not a variable or type.	<pre>#define MAX_BUFFER 1024 // good: macro const int MAX_SCORE{100}; // bad: not a macro</pre>	NL.9: ALL CAPS only for macros
NL.20; Don't place two statements on the same line One statement per line keeps code easy to scan, step through in a debugger, and diff cleanly.	<pre>int x{1}; int y{2}; // bad int x{1}; // good int y{2};</pre>	NL.20: One statement per line

SL: The Standard Library

The standard library is well-tested and well-optimized; reach for it before writing your own version.

Rule & Explanation	Code Example	Core Guideline Ref & URL
SL.con.2; Prefer std::vector by default vector is cache-friendly and simple. Reach for another container only when you have a proven reason.	<pre>int arr[100]; // bad: fixed C array std::vector<int> v; // good: grows, safe</pre>	SL.con.2: Prefer std::vector

NR: Non-Rules and Myths

Common "rules" that sound reasonable but the guidelines explicitly reject; don't apply these blindly.

Rule & Explanation	Code Example	Core Guideline Ref & URL
NR.1; Don't force all declarations to the top The old C convention of declaring every variable at function start is not required in C++ ; declare a variable where it's first needed.	<pre>// not required in modern C++: int a, b, c; // bad: all up front // good: declare where needed int a{10}; int b{g(a)};</pre>	NR.1: No forced top-of-function declarations
NR.2; Don't insist on one return per function A single mandatory exit point isn't a real safety rule in C++ ; an early return can make code clearer, not worse.	<pre>bool IsValid(int value) { if (value < 0) return false; // fine return value < 100; }</pre>	NR.2: No single-return rule